IMOVIES

MOVIE TICKET BOOKING MADE SIMPLE

Project Description:

IMovies is a web-based movie ticket booking application that makes discovering movies and reserving seats simple and convenient. Users can browse currently running and upcoming movies, view movie details and show timings, choose a theatre and available seats, review ticket prices, and confirm a booking. The application stores movie, theatre, show, seat availability, user, and booking information. Administrators can manage movie listings and show schedules. The goal is to provide a responsive, clear, and reliable ticket-booking experience.

Scenario:

A customer opens IMovies, browses movies, selects a movie, theatre, date and showtime, and chooses seats from the seat layout. After reviewing the booking summary, the customer confirms the reservation and receives a booking reference. The system rechecks seat availability before confirming the booking. Administrators can update movie and show information when schedules change.

System Requirements:

The following software and hardware prerequisites are recommended for development, testing, and use of the IMovies Web Application.

1. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Node.js (v18 or above):** Runtime for development tools and backend server logic.
- **npm:** Installs React, Express, and related libraries.
- **React.js:** Builds the responsive user interface.
- **Browser:** Latest Chrome, Firefox, or Microsoft Edge.

System Requirements (Continued)

1. Software Requirements (continued)

- **Express.js:** REST API and backend request handling.
- **MongoDB:** Stores users, movies, theatres, screens, shows, and bookings.
- **Mongoose:** Schema-based MongoDB modeling and validation.
- **Postman:** Tests backend API requests.
- **Visual Studio Code:** Code editor and terminal.
- **Git & GitHub:** Version control and collaboration.
- **Payment gateway (optional):** Stripe or Razorpay sandbox may be used for test payments.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent.
- **RAM:** Minimum 8 GB; 16 GB recommended.
- **Storage:** At least 2 GB free space for project files, dependencies, and database data.
- **Display:** 1366 × 768 or higher.

Project Scope

Core features include movie discovery, show selection, seat selection, booking confirmation, booking history, and administration. Online payment, email/SMS notifications, reviews, and recommendations may be added as optional extensions.

Suggested Collections

Collection	Purpose
users	User profile and authentication
movies	Title, description, genre, language, duration, poster
theatres / screens	Cinema and screen details
shows	Movie, theatre, screen, date/time, and price
bookings	User, show, seats, amount, status, reference
payments (optional)	Payment provider reference and status

Epic 1: Project Architecture — 1 hour

Project architecture defines the components, their interactions, and the technologies used to deliver IMovies. The following is a suggested architecture.

Technical Architecture

In-detail explanation

- **Client:** React pages show movie listings, movie details, showtimes, seat selection, booking summary, login, and admin screens.
- **API:** Express routes receive requests, validate inputs, and forward requests to controllers.
- **Business logic:** Controllers handle movie search, show selection, booking, and cancellation rules.
- **Data:** Mongoose models store and retrieve data from MongoDB.
- **Seat consistency:** The server must re-check availability when a booking is confirmed.

Main Modules

Module	Responsibilities
Authentication	Register/sign in and protect user/admin routes
Movie Catalogue	Browse, search, filter, and view movie details
Show Management	Manage theatres, screens, dates, showtimes, and prices
Seat Selection	Display available, selected, and unavailable seats
Booking	Validate seats, calculate total, create confirmation
Admin	Manage movies, theatres, shows, and booking records

ER Diagram

The entity-relationship model shows the main data entities and their relationships. This is a proposed data model; the final schema can vary by implementation.

Core Entities

USER userId, name, email	BOOKING bookingId, seats, status	SHOW showId, dateTime, price	MOVIE movieId, title, duration

Supporting Entities

THEATRE theatreId, name	SCREEN screenId, name	PAYMENT (optional) paymentId, status

Relationships and explanation

- One **User** can create many bookings; each booking belongs to one user.
- One **Movie** can have many shows; each show refers to one movie.
- A **Theatre** contains screens, and screens host shows at different times.
- One **Show** can have many bookings; each booking records selected seat identifiers.
- A booking may have an associated **Payment** record if online payment is enabled.

Important: seat identifiers must be consistent for each screen/show, and availability must be checked on the server before confirming a booking.

User Flow

The user flow describes the path from opening the application to receiving booking confirmation.

Customer Booking Flow

Open IMovies	Browse / Search Movie	Choose Theatre & Show	Select Seats	Review Total	Confirm Booking

In-detail explanation

- Browse movies and open a movie details page.
- Choose a theatre, date, and showtime.
- Select available seats using the seat-map legend.
- Review the calculated total and booking details.
- The server validates seats again, creates the booking, and returns a booking reference.
- The user can view booking history and reservation details.

Features

- Responsive movie listing with search and genre/language filters.
- Movie details including synopsis, duration, language, rating, and poster.
- Theatre and showtime selection by date.
- Seat selection and total-price calculation.
- Booking confirmation, reference, and booking history.
- Authentication and protected account pages.
- Admin management of movies, theatres, and shows.
- Optional payment and notification integration.

Roles and Responsibilities

Role	Responsibilities
Customer	Browse movies, select shows/seats, book tickets, view history
Administrator	Maintain movie catalogue, theatres, screens, schedules
Developer	Build UI/API, model data, test flows, configure and document

MVC Architecture Pattern

The backend can use the Model-View-Controller (MVC) pattern to separate data access, business logic, and request handling. React is the user-facing view; Express routes are the API entry point.

Model Layer (Data Layer)

The Model layer defines data structures, relationships, validation rules, and constraints for MongoDB. Mongoose can define schemas and perform database operations.

- Define models for User, Movie, Theatre, Screen, Show, Booking, and optional Payment.
- Perform Create, Read, Update, and Delete (CRUD) operations.
- Validate required fields, unique identifiers, seat data, and status values.
- Maintain references such as movieId, theatreId, showId, and userId.

Controller Layer (Business Logic Layer)

Controllers process request data, apply application rules, call model operations, and return responses.

- List movies, retrieve details, and search/filter the catalogue.
- Check show information and calculate booking totals.
- Re-check seat availability and create bookings safely.
- Restrict catalogue management to administrators.
- Return consistent success messages and errors.

View Layer (Client UI)

The React frontend provides screens and interaction states. Components may include MovieCard, MovieDetails, ShowSelector, SeatMap, BookingSummary, LoginForm, MyBookings, and AdminDashboard.

This separation improves readability, testing, maintenance, and future expansion.

Request Flow in MVC Architecture

A typical IMovies request follows these steps: 1) The user selects a movie or show in React. 2) The client sends an HTTP request to an Express endpoint. 3) The route runs middleware such as authentication and validation. 4) The controller processes the request and calls model operations. 5) The model reads or updates MongoDB. 6) The controller returns a JSON response. 7) React updates the interface.

Example: Confirm Booking

React UI	POST /api/bookings	Booking Controller	Check Seats + Save	JSON Confirmation

Suggested API Endpoints

Method	Endpoint	Purpose
GET	/api/movies	List/search movies
GET	/api/movies/:id	Get movie details
GET	/api/shows?movieId=...&date;=...	Find shows by movie/date
GET	/api/shows/:id/seats	Get seat availability
POST	/api/bookings	Create booking after validation
GET	/api/bookings/my	Get signed-in user's bookings
PATCH	/api/bookings/:id/cancel	Cancel booking if allowed
POST	/api/auth/register	Register a user
POST	/api/auth/login	Authenticate user
POST	/api/admin/movies	Create movie (admin only)

Security note: protect private endpoints, validate inputs on the server, keep secrets in environment variables, and never store raw payment-card details in the application database.

Epic 2: Project Setup and Configuration — 30 minutes

Creating the project folder

1. Create a folder named **imovies**. 2. Inside it, create **client** and **server**. 3. Open the main folder in Visual Studio Code.

Suggested folder structure

```
imovies/
  client/src/components/
  client/src/pages/
  client/src/services/
  server/models/
  server/controllers/
  server/routes/
  server/middleware/
  server/config/
  server/server.js
```

Client setup (React app)

Open a terminal and create the React application with Vite:

```
cd client
npm create vite@latest . -- --template react
npm install
npm run dev
```

Open the local URL printed by Vite (usually http://localhost:5173).

Server setup (Express)

```
cd server
npm init -y
npm install express mongoose cors dotenv
npm install --save-dev nodemon
```

Create server.js and folders named models, controllers, routes, middleware, and config. Add a development script such as **dev: nodemon server.js**.

Environment configuration

Create a .env file in the server folder and do not commit it to version control.

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/imovies
JWT_SECRET=replace_with_a_local_secret
```

Epic 3: Backend Development — 8 hours

Backend implementation checklist

- Configure Express, CORS, JSON parsing, environment variables, and central error handling.
- Create routes/controllers for authentication, movies, theatres, shows, seats, and bookings.
- Validate IDs, required fields, show dates, selected seats, and booking status.
- Add authentication middleware and role checks for administrator endpoints.
- Return consistent HTTP status codes and JSON responses.
- Re-check seat availability on the server before confirming a booking.

Reference screenshots / recording

Add screenshots or a narrated recording showing backend routes, controllers, middleware, and successful API tests. Explain every third-party API used and identify optional features.

Epic 4: Database Development — 2 hours

Database implementation checklist

- Create the imovies database and Mongoose schemas for users, movies, theatres, screens, shows, and bookings.
- Add required fields, timestamps, and indexes such as unique email and booking reference.
- Use consistent seat identifiers and associate every booking with a show.
- Prepare sample movie, theatre, and show data for testing.
- Test CRUD operations and confirm that references are valid.

Suggested Booking fields

Field	Description
bookingReference	Unique confirmation reference
userId / showId	References to customer and show
seats	Array of selected seat identifiers
totalAmount	Calculated booking total
status	Pending, confirmed, cancelled, or defined state
createdAt	Booking creation timestamp

Epic 5: Front-End Development — 8 hours

Frontend implementation checklist

- Create Home/Movies, Movie Details, Show Selection, Seat Selection, Booking Summary, Login/Register, My Bookings, and Admin pages.
- Build reusable components: Navbar, MovieCard, SearchBar, ShowTimeButton, SeatMap, Loader, and ErrorMessage.
- Connect React pages to Express APIs using a shared service module.
- Show loading, empty, error, selected-seat, and booking-success states.
- Make layouts responsive for desktop, tablet, and mobile.
- Test keyboard navigation, readable labels, and clear seat-state legends.

Reference screenshots / recording

Include screenshots of the home page, movie details, showtime selection, seat map, booking summary, confirmation, and admin screens. Explain external APIs used for posters, payments, or notifications.

Epic 6: Project Execution — 30 minutes

Step 1: Start the frontend

```
cd client
npm install
npm run dev
```

Open the Vite URL shown in the terminal, usually http://localhost:5173.

Step 2: Start the backend

```
cd server
npm install
npm run dev
```

Step 3: Configure environment variables

Ensure server/.env contains a valid MongoDB connection string and required authentication secret. Never upload real secrets to a public repository.

Step 4: Verify the application

· Confirm the backend starts on the configured port, for example http://localhost:5000.

- Test movie listing, show selection, seat availability, booking creation, and booking history.
- Try booking the same seat from two sessions and verify conflicting confirmed bookings are prevented.

Project Deliverables

Output Screenshots with Explanation

- Home/movie listing — explain search, filters, and movie cards.
- Movie details — explain movie information and showtimes.
- Show and seat selection — explain seat legend and availability.
- Booking summary and confirmation — explain selected seats, total price, and booking reference.
- My Bookings — explain how users review reservations.
- Admin screens — explain movie, theatre, and show management.

Demo Video Link

Add a link to a short demonstration showing the journey from movie discovery to booking confirmation.

Code Drive / Repository Link

Add the GitHub repository or shared-drive link containing source code, setup instructions, and a sample environment file without secrets.

Final Testing Checklist

- Frontend and backend start successfully.
- Database connection works and sample data is available.
- Invalid inputs and unavailable seats produce clear errors.
- Users can only view their own booking information.
- Admin-only routes are protected.
- No secret keys or real payment details are committed to source control.

End of Project Documentation